

Great. Let's begin **Chapter 1 – Part 1** of the **Company Brain Engineering Handbook**.

Company Brain Engineering Handbook

Chapter 1 – Enterprise Architecture (Part 1)

Building the Foundation of Company Brain

Table of Contents

1. What is Company Brain?
 2. High-Level Architecture
 3. Complete Request Lifecycle
 4. Why Microservices?
 5. FastAPI Architecture
 6. Amazon S3 Storage
 7. PostgreSQL Architecture
 8. Redis Internals
 9. Celery Internals
 10. Worker Lifecycle
 11. Module 1 → Module 2 Integration
-

1. What is Company Brain?

Before writing any code, we must answer one question:

What are we actually building?

Most people think Company Brain is an AI chatbot.

It is **not**.

The chatbot is only the user interface.

Company Brain is actually an **Enterprise Intelligence Platform**.

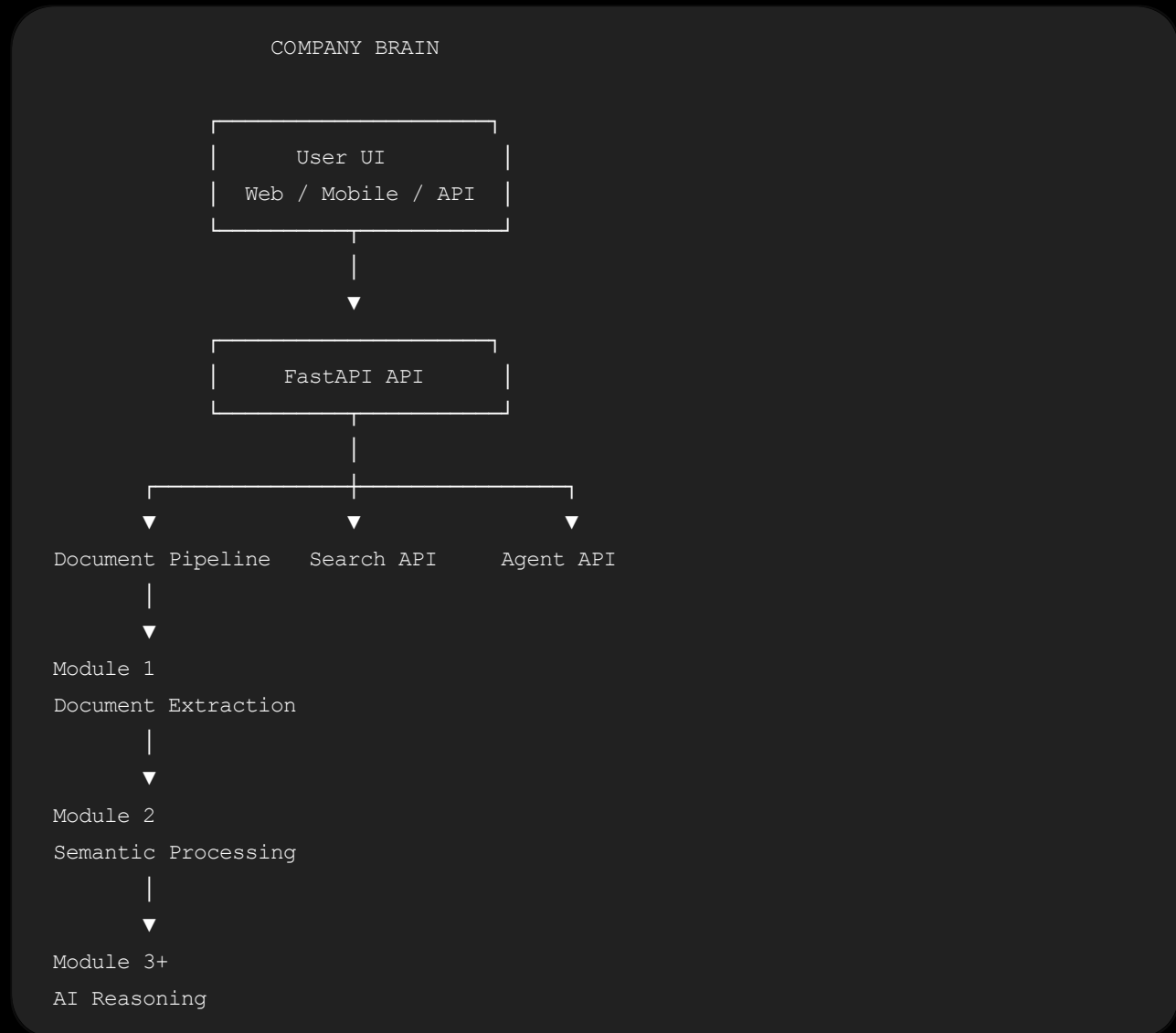
Think of it like the human brain.

A human brain has different parts:

- Eyes collect information.

- Ears collect information.
- Memory stores knowledge.
- Reasoning makes decisions.
- Speech communicates answers.

Company Brain follows the same concept.



The chatbot only sends questions.

The real intelligence happens inside the platform.

2. High-Level Architecture

Let's imagine a company uploads a single document.

Financial_Report.pdf

What actually happens?

Most beginners imagine:

Upload

↓

Extract

↓

Store

↓

Done

That is **not** how enterprise systems work.

Instead:

User

↓

FastAPI

↓

Amazon S3

↓

PostgreSQL

↓

Redis

↓

Celery Worker

↓

Module 1

↓

PostgreSQL

↓

Celery

↓

Module 2

↓

pgvector

↓

Search Engine

↓

AI Agent

Notice something.

Every component has only **one responsibility**.

This is called the **Single Responsibility Principle**.

Why not put everything inside FastAPI?

Suppose uploading one PDF requires

- OCR
- Table Detection
- Image Processing
- Embeddings
- Chunking

Time required:

8 minutes

Would you make the user wait?

No.

Instead:

Upload

↓

Return Success (2 seconds)

↓

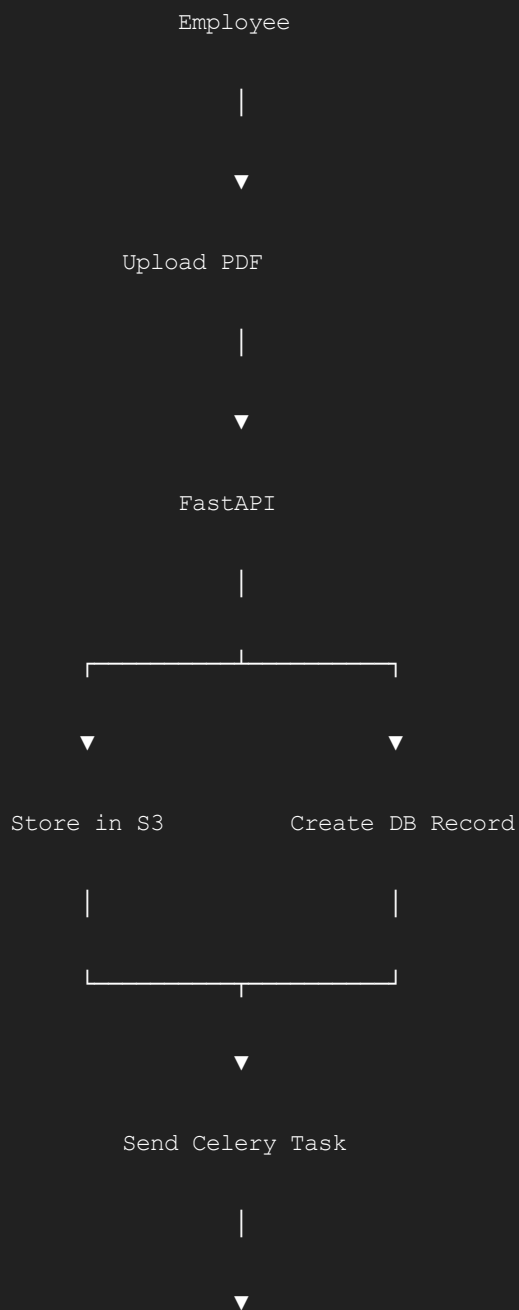
Background Processing

↓

Ready Later

That is why enterprise systems use background workers.

Complete High-Level Flow





3. Request Lifecycle

Let's follow one request.

User uploads

Financial_Report.pdf

Step 1

Browser sends

POST /documents/upload

to FastAPI.

FastAPI receives

UploadFile

object.

What is UploadFile?

FastAPI uses

Python

Run

```
from fastapi import UploadFile
```

Internally

Browser



HTTP Request



```
Multipart/Form-Data
```

```
↓
```

```
UploadFile
```

FastAPI does not load the entire PDF into RAM.

Instead

it streams it.

Advantages

- lower memory
- handles large files
- supports multiple uploads

Why FastAPI?

We choose

```
FastAPI
```

Libraries

Python

Run

```
fastapi
```

```
uvicorn
```

```
pydantic
```

```
sqlalchemy
```

```
boto3
```

```
redis
```

```
celery
```

Why?

FastAPI

- async

- high performance
 - automatic documentation
 - OpenAPI
 - dependency injection
 - Python ecosystem
-

Example

Python

Run

```
@app.post("/documents/upload")
async def upload_document(file: UploadFile):
```

FastAPI only coordinates work.

It does NOT

- OCR
- Chunking
- Embedding
- AI

It simply creates jobs.

4. Amazon S3

Where is the uploaded file stored?

Not PostgreSQL.

Not Redis.

Instead

Amazon S3.

Why?

Imagine

One company uploads

100 GB

of PDFs.

Should PostgreSQL store them?

No.

Databases are designed for structured data.

S3 is designed for files.

S3 Architecture

Company Brain

↓

Amazon S3

↓

Bucket

↓

tenant_55/

↓

documents/

↓

financial_report.pdf

Example

companybrain

↓

tenant_55

↓

documents

↓

101.pdf

Why S3?

Advantages

Unlimited storage

99.999999999%

Durability

Versioning

Encryption

Lifecycle Policies

Multipart Upload

Global Replication

Signed URLs

Libraries

Python

`boto3`

Run

AWS SDK

Example

Python

`s3.upload_file(...)`

Run

Download

Python

Run

```
s3.get_object(...)
```

Is S3 encrypted?

Yes.

We use

SSE-KMS

(Server Side Encryption with AWS KMS)

Flow

FastAPI

↓

Upload File

↓

Amazon S3

↓

AWS KMS Encrypts

↓

Encrypted Storage

The worker never sees encrypted bytes.

When authorized:

Worker

↓

Request File

↓

S3

↓

Decrypt Internally

↓

Return Plain PDF

The encryption/decryption is transparent to your code.

5. PostgreSQL

Now we need somewhere to store

- document metadata
- extracted text
- chunks
- users
- permissions

We use

PostgreSQL

Why?

Enterprise reliability

ACID

Transactions

JSONB

Extensions

Row Level Security

pgvector

Database Layout

Company Metadata DB

├─ tenants

- └─ users
- └─ documents
- └─ pages
- └─ sections
- └─ tables
- └─ images
- └─ chunks
- └─ audit_logs
- └─ permissions

Later

Vector DB

↓

embeddings

↓

HNSW indexes

Why not MySQL?

PostgreSQL supports

JSONB

Full Text Search

pgvector

Extensions

Advanced indexing

RLS

making it a better fit for enterprise AI systems.

Encryption

Data is encrypted

At Rest



AWS RDS Encryption

In Transit



TLS

Application



Reads plain data

The database handles encryption automatically.

6. Redis

Now comes one of the most misunderstood components.

Redis is NOT your database.

Redis is NOT your file storage.

Redis is NOT your vector storage.

Redis is only

Message Broker

Think of it as

Company Reception Desk

Employee



Reception



Take Token



Wait

Worker



Calls Next Token

Exactly like a queue.

Redis Queue

Task 1

Extract Document

Task 2

Extract Document

Task 3

Chunk Document

Task 4

Embed Document

Redis only stores

Task Name

Arguments

Queue

Priority

Retry Count

Task ID

NOT

PDF

Images

Chunks

Embeddings

Libraries

Python

Run

`redis`

`celery`

Why Redis?

Everything stored

in RAM

Millions of operations

per second

Latency

Less than 1 ms

Perfect for task queues.

7. Celery

Redis stores tasks.

Who executes them?

Celery Workers.

What is Celery?

Celery is a Distributed Task Queue.

It allows Python functions to run asynchronously.

Instead of

Python

```
extract_document()
```

Run

we write

Python

```
extract_document.delay()
```

Run

Difference

Normal Function

↓

Runs Immediately

Celery

.delay()

↓

Creates Task

↓

Redis

↓

Worker Executes Later

Libraries

Python

Run

```
celery
```

```
kombu
```

```
redis
```

Kombu is the messaging library used internally by Celery to communicate with brokers like Redis or RabbitMQ.

8. Worker Lifecycle

Workers are long-running Python processes.

You start them once:

Bash

```
celery -A app.workers worker
```

Now imagine the worker internally doing this:

Python

Run

```
while True:
```

```
    task = wait_for_task()
```

```
    execute(task)
```

The worker never exits.

It continuously waits for work.

When a task arrives:

```
extract_document(101)
```

Celery imports your Python function:

Python

Run

```
@celery.task
def extract_document(document_id):
```

and executes it.

Celery does not know how to extract PDFs.

It simply calls **your code**.

9. Module 1 → Module 2 Integration

This is one of the most important architectural decisions.

FastAPI only starts the pipeline.

```
FastAPI

↓

extract_document.delay(101)
```

The Extraction Worker then performs:

1. Read document metadata from PostgreSQL.
2. Download the PDF from S3.
3. Run the Module 1 extraction pipeline (text, OCR, tables, images, metadata).
4. Store extracted results in PostgreSQL.
5. Update the document status to `extracted`.

Finally, instead of calling Module 2 directly, it creates the next task:

Python

Run

```
chunk_document.delay(document_id=101)
```

Redis receives a new message:

```
Queue: chunk_queue

Task: chunk_document

Document ID: 101
```

A Chunking Worker, which has been waiting for tasks, receives it and starts Module 2.

This pattern is called **task chaining**.

Instead of one giant process:

Upload

↓

Extract

↓

Chunk

↓

Embed

↓

Index

you have independent stages:

FastAPI

↓

Extraction Worker

↓

Chunking Worker

↓

Embedding Worker

↓

Indexing Worker

Each stage:

- Reads the required data from storage.
- Performs one responsibility.
- Stores its output.
- Queues the next task.

This makes the system scalable, fault tolerant, and easy to monitor.

End of Part 1

In **Part 2**, we'll dive into:

- What actually happens inside the Extraction Worker.
- Module 2 internals.
- Chunking Worker implementation.
- Embedding Worker implementation.
- Queue chaining in detail.
- How retries, failures, and acknowledgements work.
- Complete sequence diagrams.
- Production-scale architecture.
- Best practices for Company Brain to support millions of documents and thousands of enterprise customers.

This foundation is the backbone of the entire Company Brain platform.